

PSTAT10 Data Science Principles

Lecture 14

John Inston 

johninston@ucsb.edu

UC Santa Barbara

Tuesday, August 27, 2024

Table of contents

- Introduction
- Data Manipulation
- Tiny Clothes Database
- Chinook Database
- Summary

Introduction

Recap: Lecture 13

👉 Last lecture we looked at databases, specifically:

- Relational Databases
- Data structure
 - Terminology (relations, tuples, domains, attributes)
 - Super, candidate, primary and foreign keys
- Data integrity

Also recall the following topics:

- The **tidyverse** library
- The **pipe** `|>` operator

Outline: Lecture 14

👉 Today we will cover the following material:

- Preliminaries
 - The relational model
 - Structured Query Language (SQL)
- Core Pieces
 - Database engines (SQLite)
 - Interfacing databases in R
 - SQL Queries

Data Manipulation

Recap - Relational Model

Last lecture we discussed the **relational model**.

- We emphasized **data structure** (relations, keys, etc)
- We discussed **data integrity**
- We did not discuss the third price: **data manipulation**

Data manipulation is a **slight misnomer** - manipulating data could mean many things.

- Defining new database schema
- Searching for data within schema objects
- Controlling use permissions

Structured Query Language (SQL)



SQL is the most common language used in storing, querying and manipulating structured data.

- Data Definition Language (DDL) - define database schema
- Data Manipulating Language (DML) - add, modify or delete data
- Data Query Language (DQL) - retrieves data

DCL & TCL



Note that in PSTAT10 we will not cover:

- Data Control Language (DCL)
- Transaction Control Language (TCL)

Database Engines and SQLite



The **database engine** is the software a DBMS uses to interface with its contents.

- It is not a database itself
- We will use the **SQLite** database engine
- **SQLite** is the **most widely used database engine worldwide**
- Phones, computers, televisions etc all use **SQLite**

Databases and R

To interface with databases using R we will use **RSQLite**.

- **RSQLite** embeds **SQLite** in R
- This grants us access to the **SQLite** functionality with R interface.
- We also install some additional helpful database packages.

```
1 install.packages("RSQLite")
2 install.packages("DBI") # database interface
3 install.packages("sqldf") # SQL dataframe
```

Database Packages

Load the required packages by running the code below:

```
1 library(RSQLite)
2 library(DBI)
3 library(sqldf)
```

If you do not have **all three libraries** loaded, code appearing later in the lecture will break.

- **RSQLite** allows us to interface with DBs using SQL code in **R**.
- **DBI** constructs the database interface
- **sqldf** allows us to manipulate **R** dataframes using SQL

Tiny Clothes Database

Loading the Database

Exercise - Loading the Database

Our initial database work in this course uses the **TinyCothesDB**.

- This database contains the orders of a small online clothing store.
- It stores **customer**, **product** and **sales** information.
- Our first step is to establish a connection to the database.

Exercise

Download the **tc_clean.sqlite** file from Canvas and put it in your **current working directory**.

Database Connections

To access the database, we need to **create a connection** to the database file. We do so using the `dbConnect()` function:

```
1 dbConnect(SQLite(), "path_to_my_db/my_db.sqlite")
```

Since we store our data in a separate `data` folder our function is:

```
1 dbConnect(SQLite(), "data/tc_clean.sqlite")
```


Database Connection Object

We cannot interface with the database using the `<SQLiteConnection>` directly. We must **save it as an object**.

```
1 tc_db <- dbConnect(SQLite(),"data/tc_clean.sqlite")
```

We now have all the pieces we need:

- Our **database connection** `tc_db`
- **SQL and database interfaces** (`DBI`, `RSQLite`, `sqldf`)

Exploring Databases

A `dbConnect` object only lets us `interface` with the database.

- It tells us `nothing` about its structure
- A useful first step (especially for a small database) is to explore:

The `dbListTables()` function returns the `relation names` in the remote database.

```
1 dbListTables(database_connection_object)
```

The `dbListFields()` function returns the `field names` of a remote relation.

```
1 dbListFields(database_connection_object, "table")
```

Tiny Clothes Database - `dbListTables()`

We apply the `dbListTables()` and `dbListFields()` functions to our `tc_db` connection object.

`dbListTables()`

```
1 dbListTables(tc_db)
```

```
[1] "Customer"      "Product"       "SalesOrder"    "SalesOrderLine"
```

- We see our database has four tables:

→ Customer

→ Product

→ SalesOrder

→ SalesOrderLine

Tiny Clothes Database - `dbListFields()`

`dbListFields()`

```
1 dbListFields(tc_db, "Customer")
```

```
[1] "CustomerId" "Name"      "Address"
```

- The attributes of the relation of interest are:
 - `CustomerId`
 - `Name`
 - `Address`

In general these functions become less useful as database and relation size increases!

Exercise - Database Exploration

Use the `dbListFields` command to determine the attributes of the `Product`, `SalesOrder` and `SalesOrderLine` tables.

- As you go through, think: which attributes could make up **primary keys** for each table?
- Which attributes could be **foreign keys** linking tables?

03:00

Solution - Database Exploration

```
1 dbListFields(tc_db, "Product")
```

```
[1] "ProductId" "Name"      "Color"
```

```
1 dbListFields(tc_db, "SalesOrder")
```

```
[1] "CustomerId" "OrderId"    "Date"
```

```
1 dbListFields(tc_db, "SalesOrderLine")
```

```
[1] "OrderId"    "ProductId" "Quantity"
```

Accessing Table Attributes

So far, we have functions to:

- Identify all database tables
- Identify header attributes of any table

The next step is actually accessing table data which will require us to write our first SQL query:

- Everything we've done so far is just R
- Both `dbListTables()` and `dbListFields()` are just R functions

SQL Query Structure

Basic SQL query structure is detailed below:

```
1 SELECT attributes
2 FROM relations
3 WHERE conditions
```

For example, the following query would return the **ProductId** and **Name** columns from all rows of **Product** where **ProductId > 1**:

```
1 SELECT ProductId, Name FROM Product WHERE ProductId > 1
```

Error: <text>:1:8: unexpected symbol

```
1: SELECT ProductId
    ^
```

R cannot interpret SQL code directly.

Interpreting SQL

```
1 dbGetQuery(database_connection_object, SQL_query)
2 dbExecute(database_connection_object, SQL_query)
```

To work with SQL queries in R, we can use the functions `dbGetQuery()` and `dbExecute()` from the **DBI** library.

- Use `dbGetQuery()` for **SELECT** queries only
- `dbGetQuery()` returns the query result as an **R dataframe**
- Queries that **alter database tables** use `dbExecute()`
- Note: `dbGetQuery()` may ostensibly work with commands that should use `dbExecute()`, but you can very quickly run into errors and compatibility issues.

Example - Running SQL Queries

Let's consider our previous query which returned the **ProductId** and **Name** columns from all rows of **Product** where **ProductId > 1**.

Using the query as an argument in the **dbGetQuery()** command yields the result of interest.

```
1 dbGetQuery(tc_db,  
2           "SELECT ProductId, Name  
3           FROM Product  
4           WHERE ProductId > 1")
```

	ProductId	Name
1	2	Pants
2	3	Socks
3	4	Socks
4	5	Shirts

Exercise - Basic Queries

SELECT queries are the bread and butter of SQL!

You're going to be writing several of them in the next few assignments, so write two for practice now.

- Write and execute a query to return a dataframe containing the **CustomerId, Name** and **Address** values of all customers whose addresses are either **"State"** or **"Ocean"**.
- Write and execute a query to return the **Date** values of all sales orders where **OrderId** is greater than 5.

04:00

Solution - Basic Queries

```
1 dbGetQuery(tc_db,  
2     "SELECT CustomerId, Name, Address  
3     FROM Customer  
4     WHERE Address = 'State' or Address = 'Ocean'")
```

	CustomerId	Name	Address
1	1	Alex	State
2	3	Carol	Ocean

```
1 dbGetQuery(tc_db,  
2     "SELECT Date  
3     FROM SalesOrder  
4     WHERE OrderId > 5")
```

	Date
1	8/16/19
2	10/12/19

Improving Queries

Our queries so far have been **very basic**, we are working with a very basic database.

Let's see how queries translate to a much more **complex** database.

Before connecting to the new database, **close the old connection** using the code below.

```
1 dbDisconnect(tc_db) # no outputs; closes database connection
```

Chinook Database

Chinook Database

For the remaining exercises we will use the **Chinook** database. This database is a useful training tool for several reasons:

- It is **fairly complex** as it consists of **11 interconnected relations**
- The **structural complexities** discussed in previous lectures are on full display
- The database contains **lots of data**; its tables span thousands of tuples

Download the **Chinook_Sqlite.sqlite** file from Canvas. Put it in your **working directory**.

Save it to an object using the code below.

```
1 chinook_db <- dbConnect(SQLite(), "data/Chinook_Sqlite.sqlite")
```

Exploring Complex Databases

Exploring tables is usually an appropriate first step.

```
1 dbListTables(chinook_db)
```

```
[1] "Album"      "Artist"     "Customer"    "Employee"  
[5] "Genre"      "Invoice"     "InvoiceLine" "MediaType"  
[9] "Playlist"   "PlaylistTrack" "Track"
```

Several of these tables are much larger than those in `tc_db`.

```
1 dbListFields(chinook_db, "Customer")
```

```
[1] "CustomerId" "FirstName"   "LastName"    "Company"     "Address"  
[6] "City"       "State"       "Country"     "PostalCode"  "Phone"  
[11] "Fax"        "Email"       "SupportRepId"
```


More than Dataframes

Running a SQL query through `dbGetQuery` returns a **dataframe**:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT CustomerId, FirstName, LastName, Country  
4   FROM customer  
5   LIMIT 7"  
6 )
```

	CustomerId	FirstName	LastName	Country
1	1	Luís	Gonçalves	Brazil
2	2	Leonie	Köhler	Germany
3	3	François	Tremblay	Canada
4	4	Bjørn	Hansen	Norway
5	5	František	Wichterlová	Czech Republic
6	6	Helena	Holý	Czech Republic
7	7	Astrid	Gruber	Austria

Note that the `LIMIT` argument functions similarly to the second argument of `head()` in R.

PRAGMA Command

The **PRAGMA** command is a **special tool** for **SQLite**.

It is an SQL extension specific to SQLite and used to:

1. **Modify the operation** of the SQLite library; or
2. To query the SQLite library for **internal (non-table) data**.

Queries with **PRAGMA** can get **quite complex!**

We will mainly work with

- **PRAGMA table_info(table)**
- **PRAGMA foreign_key_list(table)**

Example - PRAGA Command

```
1 dbGetQuery(chinook_db,  
2           "PRAGMA table_info(Customer)")
```

	cid	name	type	notnull	dflt_value	pk
1	0	CustomerId	INTEGER	1	NA	1
2	1	FirstName	NVARCHAR(40)	1	NA	0
3	2	LastName	NVARCHAR(20)	1	NA	0
4	3	Company	NVARCHAR(80)	0	NA	0
5	4	Address	NVARCHAR(70)	0	NA	0
6	5	City	NVARCHAR(40)	0	NA	0
7	6	State	NVARCHAR(40)	0	NA	0
8	7	Country	NVARCHAR(40)	0	NA	0
9	8	PostalCode	NVARCHAR(10)	0	NA	0
10	9	Phone	NVARCHAR(24)	0	NA	0
11	10	Fax	NVARCHAR(24)	0	NA	0
12	11	Email	NVARCHAR(60)	1	NA	0
13	12	SupportRepId	INTEGER	0	NA	0

```
1 dbGetQuery(chinook_db,  
2           "PRAGMA foreign_key_list(Customer)")
```

	id	seq	table	from	to	on_update	on_delete	match
1	0	0	Employee	SupportRepId	EmployeeId	NO ACTION	NO ACTION	NONE

Remember that a **foreign key points to the primary key** of another table.

Here **SupportRepId** in **Customer** points to **EmployeeId** in **Employee**.

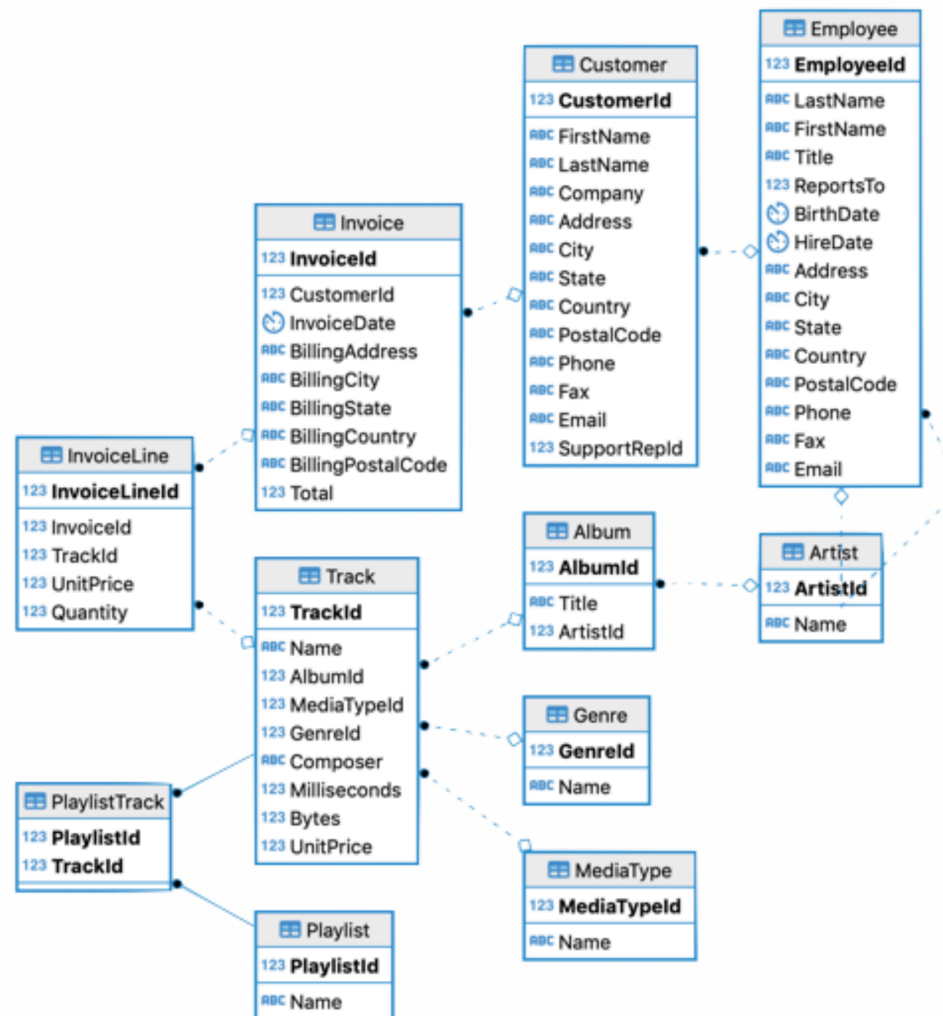
foreign_key_list()

```
1 dbGetQuery(chinook_db,  
2           "PRAGMA foreign_key_list(Customer)")
```

	id	seq	table	from	to	on_update	on_delete	match
1	0	0	Employee	SupportRepId	EmployeeId	NO ACTION	NO ACTION	NONE



Dataframe Structure



Data Integrity in Practice

So far we have only worked with **SELECT** (i.e. DQL queries)

- We can use **INSERT** to add to a table
- This is a modification to the table, so we use **dbExecute()**.

```
1 dbExecute(chinook_db,  
2           "INSERT INTO Customer (CustomerId, FirstName, LastName, Email)  
3           VALUES (97, 'John', 'Inston', 'johninston@ucsb.edu')")
```

[1] 1

```
1 dbGetQuery(chinook_db,  
2           "SELECT FirstName, LastName, Email  
3           FROM Customer  
4           WHERE CustomerId = 97")
```

	FirstName	LastName	Email
1	John	Inston	johninston@ucsb.edu

Data Integrity Table Modification

We must satisfy **entity integrity** when modifying tables.

- Recall that primary key values must be **unique**.

```
1 dbExecute(chinook_db,  
2           "INSERT INTO Customer (CustomerId, FirstName, LastName, Email)  
3           VALUES (97, 'Blythe', 'King', 'blytheking@pstat.ucsb.edu')")
```

Error: UNIQUE constraint failed: Customer.CustomerId

- Note that **SQLite** allows **NULL** primary key values, **violating entity integrity**.

```
1 dbExecute(chinook_db,  
2           "INSERT INTO Customer (CustomerId, FirstName, LastName, Email)  
3           VALUES (NULL, 'Blythe', 'King', 'blytheking@pstat.ucsb.edu')")
```

Referential Integrity in Practice

We must also satisfy **referential integrity** when modifying tables.

- Recall that foreign keys must either **point to an existing value** or be **NULL**.
- **SupportRepId** is the **Customer** foreign key for **EmployeeId**

```
1 dbExecute(chinook_db,  
2           "INSERT INTO Customer (CustomerId, FirstName, LastName, Email, SupportRepId)  
3           VALUES (92, 'Blythe', 'King', 'blytheking@pstat.ucsb.edu', 16384)")
```

```
[1] 1
```

Referential integrity is **not enforced by default**.

- To enforce referential integrity, run the following:

```
1 dbExecute(chinook_db,  
2           "INSERT INTO Customer (CustomerId, FirstName, LastName, Email, SupportRepId)  
3           VALUES (90, 'Blythe', 'King', 'blytheking@pstat.ucsb.edu', 16384)")
```

```
[1] 1
```


Cleaning Up the Mess

Let's look at what we have added to our database.

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT CustomerId, FirstName, LastName, Email  
4   FROM Customer  
5   WHERE FirstName = 'Blythe' OR LastName = 'Inston'"  
6 )
```

	CustomerId	FirstName	LastName	Email
1	90	Blythe	King	blytheking@pstat.ucsb.edu
2	92	Blythe	King	blytheking@pstat.ucsb.edu
3	97	John	Inston	johninston@ucsb.edu

DELETE Syntax

Be **very careful** if you decide to use the **DELETE** syntax as it does not allow you to undo mistakes and you will have to redownload the database.

In our case we delete the individuals that we added using the following code (you do not need to run this):

```
1 dbExecute(chinook_db,  
2   "DELETE FROM Customer  
3   WHERE FirstName = 'Blythe' OR LastName = 'Inston')"
```

[1] 3

Final Checks

We perform a final check to ensure that we have left the database as we started:

```
1 dbGetQuery(  
2   chinook_db,  
3   "SELECT CustomerId, FirstName, LastName, Email  
4   FROM Customer  
5   WHERE FirstName = 'Blythe' OR LastName = 'Inston'"  
6 )
```

```
[1] CustomerId FirstName  LastName  Email  
<0 rows> (or 0-length row.names)
```

Wrapping Up

For the next few lectures, you should be thinking about **organization**.

- It is very easy to run into issues with database connections
- Keep your database connection file somewhere logical
- Close your connections after use!

```
1 dbDisconnect(chinook_db)
```

Summary

Topics Covered

🤔 Today we looked at:

- Data Manipulation;
- Tiny Clothes Database; and
- Chinook Database.

Next Class

👉 Next class we will continue our exploration of databases and SQL with:

- Complex data selection
- SQL functions
- SQL syntax and convention

Tomorrow's lecture is code / syntax heavy; similar to early R lectures!